

```
/*
```

```
=====
```

Dart Programming Coding Assignment – Set 2

Course: Web Technology

Instructions:

1. DO NOT change function names
2. DO NOT modify the driver() function
3. Implement logic only inside the marked sections
4. Use Dart concepts wherever required

```
=====
```

```
*/
```

```
// ----- Dummy Data -----
```

```
List<int?> values = [10,null,-5,8,15,null,6,3];
```

```
class Order {
```

```
    String product;
```

```
    int orderId;
```

```
    double price;
```

```
    String status;
```

```
    String city;
```

```
    Order(this.product,this.orderId,this.price,this.status,this.city);
```

```
}
```

```
Order order = Order("Phone",501,25000,"Delivered","Mumbai");
```

```
// =====
```

```
// Q1
// Given a list of nullable integers.
//
// Rules:
// - If value is null → add 2 to sum
// - If value is negative → add its absolute value
// - Otherwise add the number itself
// =====
int computeScore(List<int?> arr){

    // WRITE YOUR CODE HERE

}

```

```
// =====
// Q2
// Using loop and conditions,
// count how many numbers are greater than 5.
// Ignore null values.
// =====
int countLarge(List<int?> arr){

    // WRITE YOUR CODE HERE

}

```

```
// =====
// Q3
// Create a function using switch
// to convert temperature.

```

```

//
// operations:
// "CtoF"
// "FtoC"
//F = (C X 1.8) + 32
//C = F - 32/1.8
// =====
double convertTemp(double value, String type){

    // WRITE YOUR CODE HERE

}

// =====
// Q4
// Return string:
//
// "Order 501 for Phone was delivered in Mumbai"
// =====
String orderDetails(Order o){

    // WRITE YOUR CODE HERE

}

// =====
// Q5
// Using if/else:
//
// Delivered -> "Order Completed"

```

```

// otherwise -> "Order Pending"
// =====

String orderStatus(Order o){

    // WRITE YOUR CODE HERE

}

// =====
// Q6
// Demonstrate inheritance.
//
// Create subclass ExpressOrder
// that extends Order and adds
// property deliveryCharge.
//
// Return the delivery charge.
// =====
double expressCharge(){

    // WRITE YOUR CODE HERE

}

// =====
// Q7
// Demonstrate mixin.
//
// Create mixin Logger with method:
//

```

```
// log() -> "Order Logged"
//
// Apply it to a class and return message.
// =====
String loggingDemo(){

    // WRITE YOUR CODE HERE

}
```

```
// =====
// Q8
// Demonstrate variable scope.
//
// Create a variable inside the function
// called tax = 18 and return it.
// =====
int taxScope(){

    // WRITE YOUR CODE HERE

}
```

```
// =====
// Q9
// Create a Future that resolves AFTER 2 seconds.
//
// If total of numbers > 20
// return "High Score"
// otherwise "Low Score"
```

```

// Ignore null values.
// =====
Future<String> evaluateValues(List<int?> arr){

    // WRITE YOUR CODE HERE

}

// =====
// Q10
// Consume evaluateValues() using async/await
// and print the result in lowercase.
// =====
Future<void> valueReport(List<int?> arr) async{

    // WRITE YOUR CODE HERE

}

// =====
// DRIVER FUNCTION (DO NOT MODIFY)
// =====
void driver() async {

    print("Q1: ${computeScore(values)}");

    print("Q2: ${countLarge(values)}");

    print("Q3: ${convertTemp(30,'CtoF')}");

```

```
print("Q4: ${orderDetails(order)}");
```

```
print("Q5: ${orderStatus(order)}");
```

```
print("Q6: ${expressCharge()}");
```

```
print("Q7: ${loggingDemo()}");
```

```
print("Q8: ${taxScope()}");
```

```
print("Q9: ${await evaluateValues(values)}");
```

```
await valueReport(values);
```

```
}
```

```
void main(){
```

```
    driver();
```

```
}
```